



Speeding up the detection of tandem repeats over the edit distance



Dina Sokol^{a,*}, Justin Tojeira^b

^a Department of Computer and Information Science, Brooklyn College of the City University of New York, 2900 Bedford Avenue, Brooklyn, NY 11210, United States

^b Department of Computer Science, The Graduate Center of the City University of New York, 365 Fifth Avenue, New York, NY 10016, United States

ARTICLE INFO

Keywords:

Tandem repeats
Lempel–Ziv factorization
Edit distance

ABSTRACT

A *tandem repeat* in a string is a sequence of two or more contiguous, approximate copies of a pattern. Finding an efficient, deterministic algorithm to perform an exhaustive search for tandem repeats in a string is an important and practical problem, due to the relevance of tandem repeats in several areas, including human identity testing, disease diagnosis, sequence homology, and population studies.

In this paper, we present an $O(nk \log^2 k + Occ)$ algorithm to find all approximate tandem repeats within a sequence of length n , allowing at most k insertions, deletions and mismatches in each repeat. Our algorithm utilizes the Lempel–Ziv factorization which was previously used in algorithms that locate exact tandem repeats, and algorithms that locate tandem repeats with only mismatches. The LZ framework is combined with speedups for calculating the edit distance, achieving a new and efficient exhaustive search for finding tandem repeats in a string.

© 2013 Elsevier B.V. All rights reserved.

1. Introduction

A *tandem repeat* is a sequence consisting of two or more adjacent copies of a pattern. *Exact tandem repeats* have identical copies of the repeated pattern, e.g. CAGCAGCAG. *Approximate* tandem repeats have some degree of deviation between the copies. Tandem repeats are one of two main types of repetitive DNA sequences, and they comprise 3% of the 3 billion bp sequence of the human genome. Identification of tandem repeats is useful for a wide variety of purposes throughout genetics. DNA repeat expansions have been linked to over thirty hereditary disorders in humans, including fragile X syndrome, myotonic dystrophy, Huntington's disease, and various spinocerebellar ataxias [9,23,32]. In addition, tandem repeats are important as genetic markers used in DNA fingerprinting [12], and are used in population studies and conservation biology [30,29].

Due to their significance in biological phenomena, there has been much effort devoted to designing algorithms and software to locate tandem repeats in a sequence. Some of the tools are based upon deterministic algorithms that exhaustively search a string for all repeats that satisfy a particular definition, e.g. mreps [14], TRed [28]. Other existing software tools are based upon constructing statistical models. This class includes: TRF (Tandem Repeats Finder) [2], ATRHunter [33], STRING [24], TandemSWAN [3], TRStalker [25], and [7]. In practice, even the algorithms that take the exhaustive search approach use some heuristical post-processor to filter and combine the found repeats. TReadS¹ is a recent web-based tool that provides a

* Corresponding author. Tel.: +1 781 951 5000 X2065.

E-mail addresses: sokol@sci.brooklyn.cuny.edu (D. Sokol), jtojeira@gmail.com (J. Tojeira).

¹ Tandem Repeat Discovery Service at bioalgo.iit.cnr.it/TReads.

tandem repeats meta-search engine. A user can run several of the above-mentioned tools, with similar parameter settings, on a given input.

One of the difficulties involved in locating tandem repeats is in accurately defining a tandem repeat. Exact repeats are clearly defined, however, when errors are introduced, it becomes necessary to define what constitutes a tandem repeat. Each of the tools for locating tandem repeats are based upon different definitions and assumptions. Thus, the output of the various tools differ, each offering insights into the presence of repeated sequences.

In this paper, we build upon the definition of the k -edit repeat [28]. We present an $O(nk \log^2 k + Occ)$ algorithm to find all maximal k -edit repeats within a sequence of length n , allowing k errors as defined by the edit distance measure, where Occ is the number of found repeats. In particular, this improves the algorithms of [18,28], reducing the $\log n/k$ factor to $\log k$. We accomplish this by using the Lempel–Ziv factorization of the input string. Previously, this technique has been used to speed up the detection of exact tandem repeats [6,4,13] and tandem repeats under the Hamming distance [15]. Dealing with insertions and deletions within the repeats, as we do in this paper, presents a much more complex problem.

The remainder of the paper is organized as follows. In Section 2 we review the underlying definition of tandem repeats over the edit distance. In Section 3 we summarize a single iteration of the algorithm for finding repeats. We describe how the Lempel–Ziv factorization is used to limit the number of iterations in Section 4. We conclude in Section 5 with a discussion of possible implementations and future work.

2. Definitions

Any definition of a tandem repeat must address two important aspects: (1) the type of mutations that occur, and (2) the way the mutations are measured. The simplest type of mutation is a mismatch, also known as a replacement or substitution. Tandem repeat finding algorithms that are based upon the Hamming distance (mismatches only) include [18,15]. The *edit* operations include insertions and deletions of characters as well, each at an equal cost. More complex models include substring inversions, weighted edit distance, and affine gap penalties, however, these measures result in algorithms with impractical time complexities. Thus, in this paper, we use the edit distance which allows us to detect a wide range of tandem repeats with highly efficient algorithms. Formally, the edit distance between two strings is defined as the minimum number of insertions, deletions, and character replacements necessary to transform one string into another.

In some cases, there is a definitive pattern, known as a *consensus* string [2,27], which each copy in the repeat must resemble. Alternatively, a repeat can be *evolutionary* [10], i.e. errors are measured between adjacent copies of the pattern. For example, gtcc gtcc gtta ctta is an evolutionary tandem repeat. Note that while adjacent copies are similar, copies at opposite ends of the repeat may bear little resemblance to each other. Conversely, any repeat that does have a consensus can also be viewed as an evolutionary tandem repeat, rendering this definition more inclusive.

In this section, we review and refine the definition of k -edit repeats [28,16]. The model of k -edit repeats uses the *edit distance* as a similarity measure, is based upon the model of *evolutionary tandem repeats*, and the error threshold k is a limit on the sum of all errors between neighboring copies. k -edit repeats can be formally defined in two different ways in a similar manner to the two possible ways of defining periodic strings (i.e. exact tandem repeats). The first definition refers to the periods of the repeat. For exact periodicity, a string s is said to be *periodic* if it can be expressed as $u^j u'$ for $j \geq 2$, u' a prefix of u . The second definition defines a periodic string in terms of its longest border, where a border of a string s is a proper prefix that is also a proper suffix of s . A string s is *periodic* if its longest border $b \geq |s|/2$. In Definition 1 we give the definition of k -edit repeats that is based upon periods and in Definition 2 we give the definition that is based upon the longest border.

The *edit distance* between two strings is defined as the minimum number of edit operations between two strings, where the edit operations include substitution, insertion and deletion of a single character. An *optimal sequence alignment* between two sequences represents an alignment whose distance is the edit distance between the two strings. Let $ed(s_1, s_2)$ denote the edit distance between the two strings s_1 and s_2 .

Definition 1. A string R is a k -edit repeat if it can be partitioned into consecutive substrings,

$R = u''u_1u_2 \dots u_\ell u'$, $\ell \geq 2$, such that $ed(u'', u'_1) + \sum_{i=1}^{\ell-1} ed(u_i, u_{i+1}) + ed(u'_\ell, u') \leq k$, where u'_1 is some suffix of u_1 and u'_ℓ is some prefix of u_ℓ .

A k -edit repeat is a sequence of evolving copies of a pattern such that there are at most k insertions, deletions, and mismatches, overall, between all consecutive copies of the repeat. For example, the word $R = \text{caagct ccagct ccgc}$ is a 2-edit repeat.

In an error-free periodic string (i.e. an exact tandem repeat), if a suffix of the string aligns against itself at location $p + 1$ then it has period size p , for e.g. in the string abcabcabcab , $p + 1$ is 4, and the period size p is 3. In this paper, we use p to denote the size of the first copy of the tandem repeat, and $p + 1$ to represent the first character of the repeat's suffix that aligns against the repeat's prefix. Since we are attempting to align a proper prefix of a string with a proper suffix of the same string, and we allow insertions and deletions, it is possible for the string to catch up with itself, aligning characters in the prefix with the exact corresponding characters in the suffix. A p -restricted sequence alignment is an edit distance alignment between two strings that disallows p insertions into the first string (alternatively, p deletions in the second string). We denote the p -restricted edit distance between two strings s_1 and s_2 as: $ed_p(s_1, s_2)$. Note that this is not symmetric, i.e. it differs from $ed_p(s_2, s_1)$.

Table 1

This table shows how the optimal sequence alignment of R_{pre} and R_{suff} can be divided into periods (or *copies*) of an evolutive tandem repeat. The variable q is introduced to represent the length of the suffix of R that is not part of R_{pre} .

$R_{pre} =$	$R_1 \dots R_{n-q} =$	u''	u_1	u_2	$\dots$	$u_{\ell-1}$	u'_ℓ
$R_{suff} =$	$R_{p+1} \dots R_n =$	u'_1	u_2	u_3	$\dots$	u_ℓ	u'

Definition 2. A string R is a k -edit repeat if and only if there exists a prefix of R , R_{pre} , and a suffix R_{suff} of R , beginning at location $p + 1$ in R , such that: $ed_p(R_{pre}, R_{suff}) \leq k$ and $|R| \leq |R_{pre}| + |R_{suff}|$.

In Table 1 we show the correspondence of the alignment of R_{pre} and R_{suff} from Definition 2, to the periods that are described in Definition 1. In [28] an algorithm is given to perform the translation from the sequence alignment of R_{pre} and R_{suff} to the repeat's copies.

Since each repeat may have many subrepeats, only maximal repeats are of interest. A k -edit repeat is *maximal* if it cannot be extended to the right or left, with matching characters, maintaining $\leq k$ errors. We point out that there are often multiple ways to break a tandem repeat into periods, several of which may be optimal in terms of total errors. This corresponds to different locations in the edit distance matrix of R_{pre} and R_{suff} that have the same minimal value. Each value can be traced back yielding a unique alignment. For our purposes, the particular alignment is not important, we are only interested in knowing that at least one such alignment exists. To illustrate with an example, we show both the sequence alignment of R_{pre} and R_{suff} and the multiple sequence alignment of the periods for a given tandem repeat.

Example 1. Repeat $R = \text{caagctccagctccgc}$, $R_{pre} = R[1 \dots 11]$, $R_{suff} = R[7 \dots 16]$, $k = 2$.

Optimal Sequence Alignment of R_{pre} and R_{suff} . (We include the entire string R on each row to show the context.)

							1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
R_{pre}							c	a	a	g	c	t	c	c	a	g	c	t	c	c	g	c
R_{suff}	c	a	a	g	c	t	c	c	a	g	c	t	c	c	-	g	c					
	1	2	3	4	5	6	7	8	9	10	11	12	13	14		15	16					

Multiple Sequence Alignment of Periods of Repeat $R = \text{caagctccagctccgc}$:

1	c	a	a	g	c	t	6
7	c	c	a	g	c	t	12
13	c	c	-	g	c		16

From an algorithmic perspective, there is a major advantage in working with Definition 2. We can obtain all the necessary information on alignments of possible R_{pre} 's and R_{suff} 's from an edit distance matrix of the entire input string against itself. The edit distance matrix can be constructed using a modified version of the Smith–Waterman dynamic programming algorithm. This is the basis of our algorithm for detecting k -edit repeats in each iteration which we describe in the following section.

3. Achieving an $O(nk \log k)$ single iteration

The algorithm that we present in this paper uses as a subroutine a procedure that searches for all repeats in a string s that include a particular character, s_c . The idea of searching for repeats that cross a point originates in [22] where it is used to find all squares in a string in linear time. It has since been extended for use with mismatches in [18,15] and a variation is used for edit distance with squares (exactly two copies) in [18], and edit distance with runs (any number of copies) in [28].

In previous work with the edit distance, the character to be included was always fixed at the midpoint of the string, i.e. $s_c = s_{n/2}$. In this paper, we generalize to find all repeats including a variable character $1 \leq c \leq n$. In order to locate all repeats that include the character s_c , it is necessary to consider all possible alignments in which s_c corresponds to another character in the string, including a gap character. In Algorithm 1 we summarize the procedure that finds all k -edit repeats that span s_c . Lines 1–15 deal with the case where location c aligns with a text position to the right² of c . These repeats necessarily have their rightmost copy beginning after position c . By symmetry, alignments for values $p < c$ can be produced. Lines 16–18 have a comment for the case where p iterates from 1 to $c - 1$.

The following lemma proves that all repeats that include s_c will be found with an optimal alignment of the characters. This is not a trivial observation since the algorithm concatenates two parts of an alignment. Yet, the concatenation of two optimal alignments does not necessarily produce an optimal alignment, as shown in the following example.

² In early literature [22] these repeats were referred to as “right repeats”.

Algorithm 1: Find Repeats that Include a Particular Character

Input: String s of length n , an integer c , $1 \leq c \leq n$, and an integer k representing the error threshold.
Output: All k -edit repeats in s that include the character s_c .

```

1 for  $p = 1$  to  $n - c$  do
2   for  $k' = 0$  to  $k$  do
3     Forward Extension: Find the longest prefix of  $s_c \dots s_n$  that  $p$ -restricted matches a prefix of  $s_{c+p} \dots s_n$  with  $k'$  errors;
4     Backward Extension: Find the longest suffix of  $s_1 \dots s_{c+p-1}$  that  $p$ -restricted matches a suffix of  $s_1 \dots s_{c-1}$  with  $k'$  errors;
5   end
6   // Consider each pair  $(k_1, k_2)$  such that  $k_1 + k_2 = k$ 
7   for  $k_1 = 0$  to  $k$  do
8      $k_2 \leftarrow k - k_1$ ;
9      $\ell_1 \leftarrow$  length of the forward extension with  $k_1$  errors;
10     $\ell_2 \leftarrow$  length of the backward extension with  $k_2$  errors;
11    if  $\ell_1 + \ell_2 \geq p$  then
12      there is a tandem repeat extending from  $s_{c-\ell_2+1} \dots s_{c+\ell_1}$ ;
13    end
14  end
15 end
16 for  $p = 1$  to  $c - 1$  do
17   // Replicate above code for  $1 \leq p < c$ , modifying indices appropriately.
18 end

```

Example 2. Concatenation of two optimal alignments yielding a non-optimal alignment.

$$\begin{array}{cccccc} c & a & - & g & g & t \\ c & a & g & - & g & t \end{array} + \begin{array}{cccccc} g & g & t & - & g & t \\ g & t & - & g & t & - \end{array} = \begin{array}{cccccc} c & a & - & g & g & t \\ c & a & g & - & g & t \end{array}$$

Lemma 1. Algorithm 1 locates all k -edit repeats in a string s that include s_c .

Proof. The proof is by contradiction. Assume that there exists a repeat R with an optimal alignment of R_{pre} and R_{suff} that includes s_c but is not found by Algorithm 1. In the optimal alignment, either s_c aligns with another character of R , or it is deleted, and aligns against a gap. First, consider the case that s_c aligns against another character, s_q . If a given point in an alignment is fixed, then the concatenation of the optimal alignment to the left of the fixed point, with the optimal alignment to the right yields an optimal alignment. Since Algorithm 1 attempts to fix every possible character with s_c , when it considers $p = q - c$, s_c is fixed against s_q and R would be found, hence a contradiction. Now, consider that s_c aligns against a gap character. In the optimal alignment, let s'_c be the first character following s_c that aligns with an actual character s_q . When calculating the forward extension with $p = q - c$, the optimal alignment will delete s_c in order to get s'_c to match s_q . Once again, since the algorithm considers every possible value of q , we have a contradiction. $\square$

Since edit distance allows deletion of characters, if there is indeed a k -edit repeat that includes s_c it will be found using several different values of p , only one of which will be optimal. Suppose p is optimal, observe that $p + i$ will give a similar alignment with i additional insertions. The optimal alignment will have the longest length for the number of errors, since it does not include any of the unnecessary insertions. Thus, we use a local maximum algorithm to detect the best p , and report only the best version of each repeat.

Time Complexity of Algorithm 1

The naive approach to implementing Algorithm 1 would be to construct a full edit distance matrix of the string s against itself. This matrix has n^2 entries, and takes $O(n^2)$ time to compute. However, this can be reduced to nk , by computing only the diagonals where it is possible to have k errors or less. Because each move away from the main diagonal raises the minimum possible errors by 1, only elements on the main diagonal and k diagonals on each side could potentially contain k or fewer errors. Additionally, any element containing more than k errors cannot be part of a path containing $\leq k$ errors. Calculating up to k errors on $O(k)$ diagonals, we are able to store the results in terms of the farthest point reachable on each diagonal d with k' errors, where $0 \leq k' \leq k$, resulting in a $k \times k$ matrix [31,19]. In addition to saving space, this allows us to take advantage of incremental string comparison [17]. In [17], Landau, Myers and Schmidt show how to get from one $k \times k$ matrix to the next in $O(k)$ time since the new matrix deals with the same strings as the previous one with the deletion of the first character of the second string. This yields overall $O(nk)$ time for computing the forward and reverse extensions (lines 2–5) for all $0 \leq k' \leq k$.

It remains to analyze lines 7–14 in Algorithm 1. In order to detect repeats with only k errors, the loop examines each combination of errors forward and backward that result in k total errors. Because the $k \times k$ matrix is updated in $O(k)$ time,

Algorithm 2: Find all k -edit Repeats within a String

Input: String s of length n LZ-factored into $f_1 \dots f_z$ and then split into adjacent non-overlapping blocks $B_0 \dots B_{z'}$, each B_i containing $k + 1$ consecutive LZ-factors.

Output: All k -edit repeats in s .

// Stage 1: Find repeats that reach block boundary

```

1 for  $i = 0$  to  $z' - 1$  do
2   Call Algorithm 1 on  $B_i B_{i+1}$ , and the  $|B_i B_{i+1}|$  preceding characters with  $s_c$  as the last character in  $B_i$ . (Discard repeats that also include last character of  $B_{i+1}$ .)
3 end

```

// Stage 2: Find repeats that reach a factor boundary

```

4 foreach  $B_i = f_{i,0} \dots f_{i,k}$  do // for each block which is, by definition,  $k + 1$  factors
5   for  $j = 0$  to  $\log k$  do
6     Split block  $B_i$  into  $\frac{k}{2^j}$  consecutive substrings, each containing  $2^j$  factors;
7     Call Algorithm 1 on each substring with  $s_c$  as the last character of factor  $\frac{2^j}{2}$  within the substring
8   end
9 end

```

// Stage 3: Find repeats contained within a single factor

```

10 for  $i = 2$  to  $z$  do // for each LZ-factor
11   Use the pointers from the LZ-factorization to locate the previous occurrence of  $f_i$  minus its last character, and copy the details about previously reported repeats.
12 end

```

determining the longest extension for each error amount k' , $0 \leq k' \leq k$, is nontrivial³ and takes $O(k)$ time to calculate. However, [18] reduces this by maintaining a heap for each k' that contains all columns with value k' . This allows the update for each k' in $O(\log k)$ time. Thus, the total complexity for an iteration of Algorithm 1 for any given c is $O(nk \log k)$.

4. Framework with Lempel–Ziv factorization

We have shown in the previous section that we can find all k -edit repeats that include a particular character in a string of length n in $O(nk \log k)$ time. In this section, we use the Lempel–Ziv factorization to bound the number of iterations. We show that the total number of necessary iterations is $O(\log k)$, yielding overall time complexity of $O(nk \log^2 k + Occ)$.

The *Lempel–Ziv factorization* (or LZ-factorization) of a string s [20,34] is the decomposition $s = u_0 u_1 \dots u_z$, where each u_i (except possibly u_z) is the shortest prefix of $u_i u_{i+1} \dots u_z$ that does not occur to the left in s . For example, given the string `abbabbabbbb` the LZ-factorization is `a . b . ba . bbabbbb`. Note that this version of the LZ-factorization allows the previous factor to extend past the start of the current factor, and thus admits copy overlap.

In other words, the LZ-factorization partitions a string into factors such that each factor is the longest prefix of the remaining string that has already occurred previously, plus one additional character. This has two important implications: (1) any given factor does not appear in its entirety anywhere to the left of itself, and (2) any prefix of any given factor must appear previously in s .

Due to its ability to discover maximal repeating occurrences in a string, the LZ-factorization is a very effective tool for discovering repeats in strings. The LZ-factorization is used in Main's algorithm [21] for computing leftmost runs, and in [13, 6] for computing all maximal runs in linear time, albeit without errors. The LZ-factorization was extended for use with errors in [15] where repetitions with hamming distance of k are found. In this paper, we show that the LZ-factorization is effective as well for locating repeats with edit distance of k .

Definition 3. We say that a repeat *reaches the boundary* between two adjacent substrings u and v , if it includes the last character of u .

Our algorithm begins by computing the LZ-factorization of the input string, $s = f_1 \dots f_z$. Then, the input string is partitioned into adjacent, non-overlapping blocks, $B_0 \dots B_{z'}$, each B_i containing $k + 1$ consecutive Lempel–Ziv factors. Repeats are then found in three stages, as outlined in Algorithm 2. In the first stage, the algorithm finds all repeats that reach a *block* boundary. In the second stage, all repeats that reach a *factor* boundary (but not a block boundary) are found. Finally, in stage 3, all repeats that are contained within a single LZ-factor are found.

³ In practice, we ignore this difficulty and simply report a single repeat with $2k$ errors. This eliminates the possibility of the same repeat being reported with several different shifts, and greatly reduces the size of the output.

4.1. Stage 1 – Repeats that reach a block boundary

In Stage 1, [Algorithm 1](#) for a single iteration is called on the input string $B_i B_{i+1}$, and the $|B_i B_{i+1}|$ preceding characters, with s_c as the last character in B_i . In this section, the first goal is to prove that every repeat that reaches a block boundary is included in one such substring. To this end, in [Lemma 4](#) we prove a bound on the length of any possible repeat in terms of block sizes. Then, in [Theorem 1](#) we prove that all repeats in the input string s that reach a block boundary are indeed found in Stage 1 of [Algorithm 2](#). Finally, in the time complexity analysis, we show that each character in the input string is processed a constant number of times.

Lemma 2. *Let R be a k -edit repeat, with prefix R_{pre} and suffix R_{suff} . R_{suff} cannot span $k + 1$ complete LZ-factors.*

Proof. The proof is by contradiction, and is based upon the pigeon-hole principle. Assume that R_{suff} spans $k + 1$ LZ-factors. Then, R_{suff} must contain at least one factor that exactly matches the corresponding substring in R_{pre} . Otherwise, each factor would contribute at least one error, resulting in $k + 1$ errors. However, this contradicts the LZ-factorization which states that a factor does not occur earlier in the same string. $\square$

Remark. We cannot say the same about R_{pre} since the Lempel–Ziv factorization only gives us information about what appears previously in the string, but tells us nothing about what may follow.

Corollary 1. *The suffix R_{suff} of any repeat R cannot contain the last character of more than one block.*

Since each block contains $k + 1$ factors, it follows from [Lemma 2](#) that no suffix of a repeat, R_{suff} , can span an entire block. If R_{suff} contained that last character of more than one block, then it would be spanning an entire block, which contradicts [Lemma 2](#).

Lemma 3. *Let B_i be the block containing the first character of R_{suff} . Then, $|R_{suff}| \leq |B_i B_{i+1}| - (k + 2)$.*

Proof. By [Corollary 1](#), a suffix of a repeat, R_{suff} , cannot reach two block boundaries, thus any suffix that starts in B_i must be entirely contained in $B_i B_{i+1}$. Furthermore, $B_i B_{i+1}$ consists of $2k + 2$ factors, while a suffix can span at most k complete factors ([Lemma 2](#)). Hence, at least some part of the remaining $k + 2$ factors will not be contained in R_{suff} . Because each factor must contain at least one character, the $k + 2$ factors contain at least $k + 2$ characters that are in $B_i B_{i+1}$ but not in R_{suff} . $\square$

Lemma 4. *A repeat R , with prefix R_{pre} and suffix R_{suff} , in which R_{suff} begins in block B_i , has length $|R| \leq 2|B_i B_{i+1}|$.*

Proof. By [Definition 2](#), the total length of a repeat $|R| \leq |R_{pre}| + |R_{suff}|$. R_{pre} must match R_{suff} with at most k insertions and deletions, yielding $|R_{suff}| - k \leq |R_{pre}| \leq |R_{suff}| + k$. Choosing the longest possible length for R_{pre} , we have, $|R| \leq 2|R_{suff}| + k$. [Lemma 3](#) shows that $|R_{suff}| \leq |B_i B_{i+1}| - (k + 2)$. Plugging this into the previous equation, we have, $|R| \leq 2(|B_i B_{i+1}| - k - 2) + k \leq 2|B_i B_{i+1}|$. $\square$

Theorem 1. *Stage 1 of [Algorithm 2](#) locates all repeats that reach a block boundary.*

Proof. We consider two distinct possible cases for a repeat that includes the last character of block B_i . Case 1 is that the last character of B_i is included in R_{suff} of a repeat. For this case, by [Lemma 3](#), R_{suff} begins in B_i and ends in B_{i+1} . By [Lemmas 1](#) and [4](#) the repeat will certainly be found in the substring of $B_i B_{i+1}$ and the $|B_i B_{i+1}|$ preceding characters.

Case 2 is that the last character of B_i is included in R_{pre} , thus it is in a repeat, but it is not included in R_{suff} . Since R_{pre} may span several blocks, it is possible that the last character of several blocks is included in R_{pre} . Since we only need to find this repeat once, it suffices to show that there is some iteration that will find such a repeat. Consider the iteration in which B_i is the last block for which R_{pre} includes the last character. Then, R_{suff} must begin in B_{i+1} . Furthermore, for repeats of Case 2, R_{suff} does not contain a last character of a block, and thus R_{suff} also ends within B_{i+1} . The iteration that considers the boundary of $B_i B_{i+1}$ will locate a Case 2 repeat since it is entirely contained in the substring of $B_i B_{i+1}$ and the $|B_i B_{i+1}|$ preceding characters. Thus, in stage 1, all repeats that reach a block boundary (i.e. include the last character of a block) will be found. $\square$

Time Complexity for Stage 1: [Algorithm 1](#) for a single iteration is called for each block boundary. In the call to [Algorithm 1](#) on block B_i , the number of characters included is $2|B_i B_{i+1}|$. Each block is included in the second half of the searched substring at most twice (once as B_i and once as B_{i+1}). The characters in the first half of the searched string may span many blocks, but they can be charged to the characters in the second half of the string since the length of the substring searched only doubles. Thus, in stage 1 each character in the input string is processed $O(1)$ (amortized) times, and overall, the complexity for this stage is the same as for a single iteration, $O(nk \log k)$.

4.2. Stage 2 – Repeats within a block that reach a factor boundary

In Stage 2, we process each block separately, searching for repeats that lie completely inside a block, do not contain the last character of any block, but reach a factor boundary. We describe the recursive view of this algorithm, which follows the idea of Main and Lorentz [[22](#)]. The Main and Lorentz algorithm splits the input string into two parts, finds repeats that cross the center, and then recursively processes each half. We use this idea, but split the string into $k/2$ and $k/2 + 1$ factors, where

k is the number of allowed errors, and search for repeats that reach the boundary (i.e. include the last character in factor $k/2$). Then, we recursively process each “half” of the string, where half is now defined in terms of number of factors and not the length of the string.

In the pseudocode for Algorithm 2, we use the iterative version of this recursion for two reasons: (1) this gives a more natural order to the finding of repeats and in the implementation it is easier to discard repeats that have been found previously, (2) the time-complexity analysis is easier when viewing this code. (To simplify the presentation in the code we assume that k is a power of 2; no difficulty arises in the general case.) Note that in lines 5–6 we use k (and not $k + 1$) since the last character of factor $k + 1$ is the last character of the block.

Time Complexity for Stage 2: This stage has $O(\log k)$ steps (or levels of recursion) and at each level Algorithm 1 is called on a set of consecutive, disjoint substrings. Recall that the complexity for Algorithm 1 is $O(nk \log k)$, thus the overall complexity of this stage is $O(nk \log^2 k)$.

4.3. Stage 3 – Repeats within an LZ-factor

In the third and final stage, we find all repeats that lie completely inside a factor (i.e. do not contain the last character of a factor). According to the definition of an LZ-factorization (see beginning of Section 4), each factor minus its last character appears to the left in the string. Thus, all substrings that need to be processed in this stage are guaranteed to have already been processed at their first occurrence in the string.

We follow the algorithm of [13] and during the computation of the LZ-factorization use the suffix tree to store a pointer to an earlier occurrence for each factor. Then, reported repeats are kept in sorted order according to their ending positions. Detecting repeats in stage 3 is done by processing factors from left to right, and copying the repeats that have occurred earlier. The complexity of this stage is $O(n + \text{Occ})$, where Occ is the number of repeats found.

Theorem 2. All k -edit repeats in a string S of length n can be found in time $O(nk \log^2 k + \text{Occ})$, where k is the error bound, and Occ is the number of found repeats.

Proof. Algorithm 2 computes the LZ-factorization, divides the factors into blocks and then finds repeats in three stages. Computing the LZ-factorization of a string can be done in linear time [11] and splitting the factorization into blocks is trivially linear. Summing the time complexities of all three stages in Algorithm 2 results in the total complexity of $O(nk \log^2 k + \text{Occ})$. $\square$

5. Conclusion

In this paper, we have presented a new and efficient algorithm for finding k -edit repeats within an input string. k -edit repeats are evolutive tandem repeats that allow up to k substitutions, insertions and deletions. A major contribution of this paper is demonstrating that the Lempel–Ziv factorization is useful with tandem repeats over the edit distance. Although previous work used the LZ-factorization for exact tandem repeats and tandem repeats with mismatches, in these situations the period size of a repeat is fixed. In the edit distance case, insertions and deletions change the problem entirely. This opens several interesting avenues of research including using these techniques for problems such as approximate periodicity with edit distance and approximate compressed matching.

In conclusion, we would like to mention a couple of points in relation to a future implementation of this algorithm. There are many different algorithms for computing the LZ-factorization of a string, several of which have been implemented. In particular, the mreps software [14] constructs the suffix tree of the input string, and then computes the LZ-factorization exactly as needed by our algorithm. The basic subroutine that finds repeats within a given iteration has also been written, and is included in the TRed software [28]. However, the TRed software uses only the midpoint as s_c , does not use suffix trees, and does not take advantage of the LMS [17] speedup. Thus, the job of coding Algorithms 1 and 2 of this paper is not trivial.

Implementing the algorithms of this paper in small space is another viable and exciting direction for the future. In recent years much work has been invested into the development of compressed self-indexes, such as the CSA (compressed suffix array) [26] and the CST (compressed suffix tree) [8]. It is possible to use the CST⁴ in both Algorithms 1 and 2 to yield small space implementations. Chen et al. [4,5] use the CSA, combined with space-efficient algorithms, to compute the LZ-factorization in small space. Al-Hafeedh et al. [1] provide an excellent survey of several efficient small space LZ-factorization implementations. Small-space tandem repeat software would be especially practical for use on large datasets such as the human genome.

Acknowledgment

This work has been supported in part by the National Science Foundation Grant DB&I 0542751.

⁴ For an implementation of the CST, see <http://www.uni-ulm.de/en/in/institute-of-theoretical-computer-science/research/sdsl.html>.

References

- [1] Anisa Al-Hafeedh, Maxime Crochemore, Lucian Ilie, Evguenia Kopylova, W.F. Smyth, German Tischler, Munina Yusufu, A comparison of index-based lempel-ziv lz77 factorization algorithms, *ACM Comput. Surv.* 45 (1) (2012) 5:1–5:17.
- [2] G. Benson, Tandem repeats finder — a program to analyze DNA sequences, *Nucleic Acids Res.* 27 (1999) 573–580.
- [3] V. Boeva, V. Makeev, M. Régnier, SWAN: searching for highly divergent tandem repeats in DNA sequences and statistical significance, in: *JOBIM'04*, IEEE Computer Society, 2004, in: *Proceedings JOBIM'04*, Montréal.
- [4] Gang Chen, Simon J. Puglisi, William F. Smyth, Fast and practical algorithms for computing all the runs in a string, in: Bin Ma, Kaizhong Zhang (Eds.), *CPM*, in: *Lecture Notes in Computer Science*, vol. 4580, Springer, 2007, pp. 307–315.
- [5] Gang Chen, Simon J. Puglisi, William F. Smyth, Lempel-ziv factorization using less time & space, *Math. Comput. Sci.* 1 (4) (2008) 605–623.
- [6] Maxime Crochemore, Lucian Ilie, Computing longest previous factor in linear time and applications, *Inform. Process. Lett.* 106 (2) (2008) 75–80.
- [7] Nevzat Onur Domanic, Franco P. Preparata, A novel approach to the detection of genomic approximate tandem repeats in the levenshtein metric, *J. Comput. Biol.* 14 (7) (2007) 873–891.
- [8] J. Fischer, V. Mäkinen, G. Navarro, Faster entropy-bounded compressed suffix trees, *Theoret. Comput. Sci.* 410 (51) (2009) 5354–5364.
- [9] J.R. Gatchel, H.Y. Zoghbi, Diseases of unstable repeat expansion: Mechanisms and common principles, *Nature Rev. Genet.* 6 (2005) 743–755.
- [10] R. Groult, M. Leonard, L. Mouchard, Speeding up the detection of evolutive tandem repeats, *Theoret. Comput. Sci.* 310 (1–3) (2004) 309–328.
- [11] D. Gusfield, *Algorithms on Strings, Trees, and Sequences — Computer Science and Computational Biology*, Cambridge University Press, 1997.
- [12] A.J. Jeffreys, DNA typing: approaches and applications, *J. Forensic Sci. Soc.* 3 (1993) 204–211.
- [13] R. Kolpakov, G. Kucherov, Finding maximal repetitions in a word in linear time, in: *Proc. of Symposium on Foundations of Computer Science (FOCS)*, 1999, pp. 596–604.
- [14] R. Kolpakov, G. Kucherov, mreps: efficient and flexible detection of tandem repeats in DNA, *Nucleic Acids Res.* 31 (2003) 3672–3678. <http://www.ioria.fr/mreps/>.
- [15] R.M. Kolpakov, G. Kucherov, Finding approximate repetitions under hamming distance, *Theoret. Comput. Sci.* 1 (303) (2003) 135–156.
- [16] Gregory Kucherov, Dina Sokol, Approximate tandem repeats, in: Ming-Yang Kao (Ed.), *Encyclopedia of Algorithms*, Springer, 2008.
- [17] G.M. Landau, E.W. Myers, J.P. Schmidt, Incremental string comparison, *SIAM J. Comput.* 27 (2) (1998) 557–582.
- [18] G.M. Landau, J.P. Schmidt, D. Sokol, An algorithm for approximate tandem repeats, *J. Comput. Biol.* 8 (2001) 1–18.
- [19] G.M. Landau, U. Vishkin, Fast parallel and serial approximate string matching, *J. Algorithms* 10 (2) (1989) 157–169.
- [20] A. Lempel, J. Ziv, On the complexity of finite sequences, *IEEE Trans. Inform. Theory* 22 (1976) 75–81.
- [21] M.G. Main, Detecting leftmost maximal periodicities, *Discrete Appl. Math.* 25 (September) (1989) 145–153.
- [22] M.G. Main, R.J. Lorentz, An $O(n \log n)$ algorithm for finding all repetitions in a string, *J. Algorithms* 5 (3) (1984) 422–432.
- [23] S.M. Mirkin, DNA structures, repeat expansions and human hereditary disorders, *Curr. Opin. Struct. Biol.* 16 (3) (2008) 351–358.
- [24] V. Parisi, V. De Fonzo, F. Aluffi-Pentini, STRING: finding tandem repeats in DNA sequences, *Bioinformatics* 19 (14) (2003) 1733–1738. <http://bioinf.dms.med.uniroma1.it/JSTRING>.
- [25] Marco Pellegrini, M. Elena Renda, Alessio Vecchio, Trstalker: an efficient heuristic for finding fuzzy tandem repeats, *Bioinformatics [ISMB]* 26 (12) (2010) 358–366.
- [26] K. Sadakane, New text indexing functionalities of the compressed suffix arrays, *J. Algorithms* 48 (2) (2003) 294–313.
- [27] Marie-France Sagot, Eugene W. Myers, Identifying satellites and periodic repetitions in biological sequences, *J. Comput. Biol.* 5 (3) (1998) 539–553.
- [28] Dina Sokol, Gary Benson, Justin Tojeira, Tandem repeats over the edit distance, *Bioinformatics* 23 (2) (2007) e30–e35.
- [29] G. Spong, L. Hellborg, A near-extinction event in lynx: do microsatellite data tell the tale? *Conserv. Ecol.* 6 (1) (2002) 15.
- [30] M.W. Uform, R.K. Wayne, Microsatellites and their application to population genetic studies, *Curr. Opin. Genetics Dev.* 3 (1993) 939–943.
- [31] E. Ukkonen, On approximate string matching, in: *Proc. Int. Conf. Found. Comp. Theor.*, in: *Lecture Notes in Computer Science*, vol. 158, Springer-Verlag, 1983, pp. 487–495.
- [32] K. Usdin, The biological effects of simple tandem repeats: lessons from the repeat expansion diseases, *Genome Res.* 18 (7) (2008) 1011–1019.
- [33] Y. Wexler, Z. Yakhini, Y. Kashi, D. Geiger, Finding approximate tandem repeats in genomic sequences, in: Philip E. Bourne, Dan Gusfield (Eds.), *RECOMB*, ACM, 2004, pp. 223–232.
- [34] J. Ziv, A. Lempel, A universal algorithm for sequential data compression, *IEEE Trans. Inform. Theory* IT-23 (3) (1977) 337–343.